Introduction to Decision Making Constructs in C#.Net

Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview

- Introduction to Decision Making Constructs
- Branching Statements in C#
- Jumping Statement in C#
- Control Flow Change Statement in C#
- Logical Operators in C#
- Ternary Operator in C#

Introduction to Decision Making Constructs in C#.Net

•Decision constructs show how to control the flow of program execution.

- **Branching Statements**
- **Jumping Statement**
- **Control Flow Change Statement**
- **Conditional**

The Branching Statements:

a. if

b. if-else

c. if-else if-else

d. nested if

Branching Statements in C#

- **The Branching Statements:**

    a. if

    b. if-else

    c. if-else if-else

    d. nested if

- **if-else:** This construct is use to work with single condition. obtain a literal Boolean value.

Syntax:

```
if (conditional expression)
{
        [execute if above condition returns true];
}
```

Example:

```
// Example 1
int x = 5;

if (x > 10)
{
   Console.WriteLine("x is greater than 10");
}
else
{
   Console.WriteLine("x is less than or equal to
10");
}
```

The if-else if-else:

This construct is use to work with multiple conditions but it has a draw back in compare with switch statement that its checks all conditions unless and until not satisfying with the condition. obtain a literal Boolean value.

Syntax:

```
if (conditional expression)
{
        [execute if above condition returns true];
}
else if(conditional expression){
        [Execute if above condition returns true];
}else{
         [Execute if above all condition returns false];
}
```

Example:

```
int x = 5;

if (x > 10)
{
   Console.WriteLine("x is greater than 10");
}
else if (x < 10)
{
   Console.WriteLine("x is less than 10");
}
else
{
   Console.WriteLine("x is equal to 10");
}
```

The Nested if statement

If within if is use to check conditions upon dependencies and priorities.

obtain a literal Boolean value.

Syntax:

```
if (conditional expression)
{
        [Execute if outer condition returns true];

        if(conditional expression){
        [Execute if nested condition returns true];
        }
        else{
         [Execute if nested condition returns false];
        }
}
else{
         [Execute if outer condition returns false];
}
```

Example:

```
int x = 5;
int y = 10;

if (x > 0)
{
    Console.WriteLine("x is positive.");

    if (y > 0)
    {
        Console.WriteLine("y is positive.");
    }
    else
    {
        Console.WriteLine("y is non-positive.");
    }
}
else
{
    Console.WriteLine("x is non-positive.");
}
```

Jumping Statement

- **Jumping Statement**

The switch Statement

The other simple selection construct offered by C# is the switch statement. As in other C-based languages, the switch statement allows you to handle program flow based on a predefined set of choices. This construct is more faster than if.

Syntax:

```
switch (variable)
{
case 1: [Expression/Statement];
break;
case 2: [Expression/Statement];
break;
default: [Expression/Statement];
break;
}
```

Example:

```
int dayOfWeek = 2;
string dayName;

switch (dayOfWeek)
{
   case 1:
      dayName = "Monday";
      break;
   case 2:
      dayName = "Tuesday";
      break;
default:
      dayName = "Invalid day";
      break;
}

Console.WriteLine("The day is: " + dayName);
```

Control Flow Statements

- **Control Flow Statement:**

go to-Label statement

go to statement is use to change control flow of program conditionally, using go to we can make a block for recursive call. This block usually nest in if statement

To know when it can stop recursion.

Syntax:

```
Label :

if(Conditional Expression)
{
   goto Label;
}else{
            Expression or Statement;
}
```

Example:

```
int x = 5;

if (x < 10)
{
    goto LessThanTen;
}
else
{
    goto GreaterThanOrEqualToTen;
}

LessThanTen:
    Console.WriteLine("x is less than 10");
    goto End;

GreaterThanOrEqualToTen:
    Console.WriteLine("x is greater than or equal to 10");
    goto End;

End:
    Console.WriteLine("End of the program");
```

- **Conditional Operators**

These operators also returns true/false.

- Short Circuiting operators [&& ||]
- Ternary Operator [Conditional Expression ? true : false]

Syntax:

```
if(Conditional Expression1 && Conditional Expression2)
{
        [Executes if all Expression returns true]
}else{
        [Executes if either Expression returns false]
}
```

```
if(Conditional Expression1 || Conditional Expression2)
{
        [Executes if either Expression returns true]
}else{
        [Executes if all Expression returns false]
}
```

Example:

```
int x = 5;
int y = 10;

if (x > 0 || y > 0)
{
    Console.WriteLine("At least one of x or y is
positive.");
}
else
{
    Console.WriteLine("Neither x nor y is
positive.");
}
```

```
bool isRainy = false;
bool isSunny = true;

if (isRainy || isSunny)
{
    Console.WriteLine("The weather is either rainy or
sunny.");
}
else
{
    Console.WriteLine("The weather is neither rainy nor
sunny.");
}
```

Ternary Operator in C#

Ternary operator [Conditional Expression ? true : false]

Ternary operator is alternative solution for if/else statement but this only useful When expression or statements are short it use to optimize length of code.

Syntax:

```
Conditional Expression ? Expression/Statement [true] : Expression/Statement [false]
```

Example:

```
int x = 5;
string result = (x > 0) ? "Positive" : "Non-positive";

Console.WriteLine(result);
```

References

https://www.tutorialspoint.com/csharp/csharp_decision_making

https://edu.gcfglobal.org/en/computerbasics/what-is-a-computer/1/

https://www.knowledgehut.com/tutorials/csharp/csharp-decision-making-statements

https://www.geeksforgeeks.org/c-sharp-decision-making-else-else-ladder-nested-switch-nested-switch/

https://www.dotnettricks.com/learn/csharp/understanding-decision-making-statements-in-csharp

https://www.w3schools.in/csharp/decision-making

https://www.codingninjas.com/codestudio/library/decision-making-in-c